CTI / v4.0.1

Clinical Trial Intelligence Workbench

Full operating & testing guide

Install. Explore. Fit. Inspect. Preserve.

Twelve screens, explicit formulas and source provenance.

Windows / macOS / Linux - Python 3.12

Local research application with an optional multi-provider AI assistant.

PROTOCOL

EVIDENCE

SCENARIO

Research decision support - not clinical or regulatory advice

Includes every-screen checks, key setup, failure fixes and v3 upgrade steps.

Published fixture preview and full local backend have different capabilities.

Manual edition: 8 October 2026 | Read docs/VERIFICATION.md for release evidence.

Contents

Use chapter bookmarks in your PDF viewer. The operating sequence, every-screen checks and research limits are all included.

About this guide . . . 3
1. What is included and what runs where . . . 4
2. Install on Windows with Python 3.12 . . . 5
3. Install on macOS or Linux . . . 6
4. Configure an AI provider key . . . 7
5. First complete research workflow . . . 9
6. Data provenance and safe imports . . . 10
7. Overview - operate and test . . . 11
8. Study Explorer - operate and test . . . 12
9. Complexity Lab - operate and test . . . 13
10. Experiments - operate and test . . . 14
11. Phase Academy - operate and test . . . 15
12. Phase Model - operate and test . . . 16
13. Data Manager - operate and test . . . 17
14. AI Assistant - operate and test . . . 18
15. Research Hub - operate and test . . . 19
16. Digital Twin - operate and test . . . 20
17. Validation Lab - operate and test . . . 22
18. Formula Studio - operate and test . . . 24
19. Exact PCI, metric and planning formulas . . . 25
20. v4 models, calibration and uncertainty . . . 27
21. How to interpret evidence honestly . . . 29
22. Export, evidence packets and backups . . . 30
23. Upgrade v3 to v4 safely . . . 31
24. Troubleshooting by symptom . . . 32
25. Developer checks and every-section acceptance . . . 33
26. Research CLI and AACT workflow . . . 35
27. API, authentication and hosting . . . 36
28. Tools, plugins and v5 development matrix . . . 37
29. Change the project and maintain the manual . . . 39

About this guide

Full operating, testing and upgrade guide - v4.0.1

This manual explains how to install the local application, operate all twelve screens, check formulas and evidence, diagnose failures, protect API keys and upgrade an existing installation. The application is a local research workbench built with Python 3.12, FastAPI, React and scikit-learn. The ZIP includes the compiled interface; a normal user does not need Node.js.

Version 4 adds Research Hub, Digital Twin, Validation Lab and Formula Studio while retaining the eight existing screens. It adds a seventh duration/risk model branch and separates hyperparameter tuning from uncertainty calibration. It does not establish prospective clinical performance.

The hosted preview is a five-record software fixture demonstration. The full local application can load 800 authentic public registry records, import your own data and fit research models. A fixture label, an unavailable prediction or an undefined metric is meaningful information. Never replace it with an invented number.

Reading routes. For first use, follow chapters 2-6 and then the twelve screen chapters. For a release check, use the acceptance checklist and developer commands. For an existing installation, start with the upgrade chapter. Scientific interpretation depends on the formula, modeling, provenance and limitations chapters as well as the controls on screen.

Current release evidence. Read `docs/VERIFICATION.md` for the exact environment, test results, deployment checks and untested boundaries of the delivered build. This guide deliberately does not repeat test counts or claim live service authentication. Read the saved manifest of your own run for cohort counts and fitted metrics.

All paths in commands are relative to the extracted project folder unless stated otherwise. Run commands in a terminal opened in that folder. PDF code blocks visually wrap long commands; copy exact command lines from the Markdown source when needed. Values such as `YOUR_DATASET_ID`, `YOUR_64_HEX_SHA256` and `YYYY-MM-DD` are placeholders that you must replace with your own verified values.

Research decision support only. Outputs are not clinical or regulatory advice, treatment recommendations, causal evidence or proof of trial success.

1. What is included and what runs where

The full installation contains the backend, source interface, compiled interface, research pipelines, launchers, tests, configuration, public starter data, preserved source responses and documentation. Data and training work without any AI provider key. The assistant is optional.

Environment	What works	What to expect
Local app at `http://127.0.0.1:8000`	Exploration, import, training, source matching, scenarios, evidence exports and optional assistant	One Python server with persistent local disk
Hosted fixture preview	Interface exploration, descriptive PCI, learning animations and fixture scenarios	Five labeled fixtures; backend actions explain their boundary
CLI research scripts	Reproducible experiments and feature extraction	New output folders; results are not automatically GUI history
Optional PostgreSQL metadata	Registry metadata when separately configured	Raw files and models still require persistent disk

The twelve screens are Overview, Study Explorer, Complexity Lab, Experiments, Phase Academy, Phase Model, Data Manager, AI Assistant, Research Hub, Digital Twin, Validation Lab and Formula Studio. Navigation may group related screens. Dataset selection applies to requests for that workspace; it does not overwrite source files.

On a clean start the local app registers `data/sample/trials.csv` as `sample-fixture`. Those five records test software behavior. They are not eligible scientific training evidence. Open Data Manager and load the real starter cohort before a full modeling workflow.

The server keeps local datasets, jobs and scenario history under `data/workbench/` by default. Restarting retains this history. Interrupted jobs are marked failed; start a new job after checking its error. Duration and phase training share one bounded in-process worker. Run one server process and one Uvicorn worker.

The source of truth for implemented capability is the delivered code and configuration, current manifests, and `docs/VERIFICATION.md`. Older remote design handoffs and roadmaps describe context or planned work. They do not turn a future feature into a working service.

2. Install on Windows with Python 3.12

1. Install 64-bit **Python 3.12** from `https://www.python.org/downloads/`. Include the Python launcher. Verify `py -3.12 --version` in Command Prompt.
2. Extract the entire delivered ZIP into a normal writable folder, for example `C:\CTI`. Do not run inside the compressed-folder viewer. Preserve the folder structure.
3. Double-click **SETUP_WINDOWS.bat**. It creates `.venv`, downloads pinned core dependencies and creates `.env` from `.env.example` if needed. Internet access is required for dependency installation.
4. Wait for **Setup complete**. If the script reports an error, read the first pip or Python error before closing the window.
5. Double-click **START_WINDOWS.bat**. Keep its terminal open. Open **http://127.0.0.1:8000** in your browser.
6. Double-click **CHECK_WINDOWS.bat** to run offline diagnostics and `pip check` when setup or launch fails.

The included `frontend/dist` means Node.js is unnecessary for normal use. Setup does not overwrite an existing `.env`. Do not copy a virtual environment from a different machine or older release.

Equivalent commands from Command Prompt, opened in `C:\CTI`:

```
py -3.12 --version
py -3.12 scripts\setup.py
.venv\Scripts\python.exe scripts\start.py
```

Stop the server with **Ctrl+C** in its terminal. Start again with the launcher. If you change `.env`, restart the server so new settings are read.

Installation acceptance check. The page loads without an error overlay; the initial dataset is labeled a fixture; `/api/v1/health` returns healthy status; CHECK_WINDOWS reports Python 3.12, core dependencies, built interface, starter data and writable storage. The doctor reports whether a key is present, but does not validate provider authentication.

Fixes. If `py` is missing, install the launcher and reopen the terminal. If Python 3.12 cannot be selected, install that exact minor version. For download failures, check internet/proxy access, disk space and the first pip error, then rerun setup. If Windows blocks a downloaded batch file, inspect and unblock the extracted file using your usual trusted-download workflow. Run in a user-writable directory.

3. Install on macOS or Linux

Install Python 3.12 and extract the full ZIP into a writable directory. From the project root:

```
python3.12 --version
bash setup.sh
bash start.sh
```

Open `http://127.0.0.1:8000`. Keep the terminal open and stop with Ctrl+C. The shell launchers anchor paths to the project root. If the launcher cannot find `python3.12`, install Python 3.12 through your operating system's supported method before retrying.

Direct equivalents and checks:

```
python3.12 scripts/setup.py
.venv/bin/python scripts/start.py
.venv/bin/python scripts/doctor.py
.venv/bin/python -m pip check
.venv/bin/python scripts/run_smoke_test.py
```

If `.venv` does not exist, setup has not completed. If a required module is absent, use the virtual-environment interpreter to install `requirements.txt`; a system `pip` may install into the wrong environment.

Acceptance check. The doctor reports ready, `pip check` reports no broken requirements, the local page and health endpoint load, and the smoke script validates the five fixtures while leaving model predictions unavailable. This verifies installation behavior, not predictive validity.

Port change. Set `API_PORT=8001` in root `.env`, restart, and browse `http://127.0.0.1:8001`. Use the URL printed by the launcher. Port conflicts must be resolved by stopping the other process or choosing another port; repeatedly refreshing the wrong URL cannot fix them.

Docker option. Copy `.env.example` to `.env` if it does not already exist, configure values privately, then run `docker compose up --build`. Persistent storage and one backend process remain necessary. Inspect `docker compose logs` on failure. Docker availability is optional; see `docs/VERIFICATION.md` for whether the supplied configuration was executed in this release environment.

4. Configure an AI provider key

The v4.0.1 assistant supports OpenAI, Gemini, Claude and another OpenAI-compatible Chat Completions service. Core data, modeling, formulas and animations need no key. Keep settings in root `.env`; unused key fields remain empty. Existing v4 `.env` files are preserved, so append new settings from `.env.example` when upgrading.

```
AI_PROVIDER=auto
AI_PROVIDER_PRIORITY=openai,anthropic,gemini,compatible
AI_AUTO_FALLBACK=true
OPENAI_API_KEY=
OPENAI_MODEL=gpt-5.4-mini
GEMINI_API_KEY=
GEMINI_MODEL=gemini-3.5-flash
ANTHROPIC_API_KEY=
ANTHROPIC_MODEL=claude-sonnet-4-6
COMPATIBLE_API_KEY=
COMPATIBLE_BASE_URL=
COMPATIBLE_MODEL=
```

1. Run setup so `.env` exists. Add whichever matching provider key you own.
2. Keep `AI_PROVIDER=auto`, save, stop/restart the backend and refresh the page.
3. Open AI Assistant, select a public study or dataset, and send a short question.
4. Read the actual provider and model above the answer. Key presence is not proof of authentication, quota or model access.

Create keys in your provider's developer console: OpenAI at `https://platform.openai.com/api-keys`, Gemini at `https://aistudio.google.com/api-keys`, Claude at `https://platform.claude.com/`. No key is supplied. Calls can incur API charges.

Provider selection and privacy

One configured provider is selected automatically. With multiple keys, Auto follows the priority order above, skipping missing configurations. This is an editable preference, not a measured best-model ranking. To prefer Gemini, place `gemini` first in AI_PROVIDER_PRIORITY. To pin Claude use `AI_PROVIDER=anthropic`. The provider dropdown can override the server choice. A directly chosen provider never silently switches. Set AI_AUTO_FALLBACK=false to disable fallback.

Auto can try the next configured provider after key/access/model errors, quota/rate limits, network errors, timeouts or service failures. The same bounded context may then reach more than one configured service. Refusals, safety blocks, empty/malformed results and ordinary bad requests are not rerouted. Each provider is attempted at most once per question; the reply identifies any fallback.

For another compatible service, supply its own COMPATIBLE_API_KEY, HTTPS API root (often ending `/v1`) and COMPATIBLE_MODEL. The app appends `/chat/completions`. Compatibility requires that exact chat protocol; an arbitrary vendor key alone cannot work. Do not append `/chat/completions` yourself or embed secrets in the URL. Native keys are only sent to their matching official endpoints.

Keys belong on the server, never in VITE variables, browser Connection settings, reports, screenshots, source control or chat. Connection settings takes the separate shared API_BEARER_TOKEN. Requests are independent and bounded; previous chat turns are not sent. OpenAI uses Responses with store=False; other providers have their own retention policies. This flag is not a universal retention guarantee.

Read `docs/AI_PROVIDERS.md` for request limits, fallback conditions, custom-provider requirements and troubleshooting. Live account access was not tested without your key. After changing any key or model, restart the backend.

5. First complete research workflow

1. Open **Phase Academy** to learn the interface without requiring data, fitted models or keys. Try Phase III, Step, Play and Reset.
2. Open **Data Manager** and click **Load starter cohort**. Confirm 800 records, authentic public source labeling and the checksum.
3. Open **Overview** and **Study Explorer**. Review missing fields and exact source values; open a study and compare two records.
4. Open **Formula Studio** and **Complexity Lab**. Check the fixed-reference PCI methodology before interpreting the score.
5. Open **Experiments**, review readiness and run a duration/risk experiment. Wait for completed status and read warnings, retained split IDs, tuning/calibration support and held-out results.
6. Open **Validation Lab** and examine the real manifest partitions. Export the manifest and held-out CSV. An undefined metric remains unavailable.
7. Open **Phase Model**, train the separate task and compare reported versus predicted phase. Review class support and confusion matrices.
8. Open **Research Hub** and inspect a selected record's source, descriptive similarity, phase context and available model evidence.
9. Open **Digital Twin**, create an explicitly hypothetical planning scenario, inspect changes and export an evidence packet.
10. Add an optional supported provider key only if you want AI Assistant. Back up your store after stopping the application.

The starter's outcome-eligible cohort is much smaller than 800 because exact observed dates are required. Large source counts do not imply large retained model cohorts. The bundled convenience subset is completed-only, latest-state and retrospective. Its validation labels are limited; calibrated probabilities may remain unavailable. This is expected when support requirements are not met.

Bundled fitted examples are separately preserved in `examples/`. They are not automatically registered as trusted runtime models in a new local database. Load the starter and train through the GUI to create a matching local run. Read the current `docs/STARTER_RESULTS.md` and versioned manifests before comparing example numbers.

6. Data provenance and safe imports

The release bundles **800 authentic ClinicalTrials.gov API v2 records**, acquired on **2026-10-07 UTC**. The selection comprises completed interventional drug trials with phase I-IV, including combined phases, in four start-date periods from 2000-2023. Each period contributes 200 records in the API's default order. This is a convenience subset, not a random or representative registry sample.

File	Role
`data/starter/trials.csv`	Canonical 800-row table
`data/starter/metadata.json`	Query, timing, mapping, source and canonical hashes
`data/starter/raw/*.json.gz`	Eight preserved original API response pages
`data/starter/cohort_check.json`	Eligibility/split audit, not a model result

Canonical SHA-256:

```
d096795e303b5a103e0aba8bd79ffe7e85609827ea810d1c08e8e0811b5496d2
```

Keep source snapshot date, data/mapping version, canonical and raw digests, dataset ID, task version and artifact version distinct. A model version is not a source date. A checksum detects changes; it does not prove publisher authenticity or baseline provenance.

CSV/Parquet imports require `nct_id` and `title`, unique IDs and valid structure. Optional numeric values must be finite and nonnegative. Dates preserve year/month/day precision; missing days are not silently filled in. Invalid or inverted dates and malformed imports fail atomically, without registering a partial dataset. Leave missing values blank rather than inventing counts or dates.

Offline replay and fresh acquisition use new directories:

```
.venv/bin/python scripts/download_starter_data.py --replay-from data/starter --output-dir
  data/replayed-starter
.venv/bin/python scripts/download_starter_data.py --output-dir data/new-starter --per-period
  200
```

Replay verifies raw and regenerated canonical hashes. A fresh live request can return revised records or a different ordering and becomes a new source snapshot. Never overwrite the frozen starter to make results easier to reproduce. Live import failures do not substitute fixture data. The five smoke-test PCI values are 25.50, 55.40, 12.60, 41.05 and 52.90 in NCT-SAMPLE-001 through 005 order; their fitted predictions remain unavailable without artifacts.

7. Overview - operate and test

Purpose. Obtain a descriptive view of the selected dataset: row count, phase/status counts, enrollment, PCI distribution and missingness. Descriptive PCI is available before training; model predictions require a compatible fitted artifact.

Operate. Choose a dataset in the global selector. Read its source label and record count. Review PCI and missingness together. Use the study list to open a record and inspect source fields. Use the report export to save the descriptive view.

Acceptance checks.

Action	Expected result
Select fixture dataset	Five records and a software-fixture label
Load/select starter	800 records with public source provenance
Compare dataset switches	Counts and lists correspond to the selected dataset
Open a study	Source values, PCI methodology and prediction status appear
View before local training	No invented duration/risk percentage
Export report in full app	Self-contained descriptive HTML downloads

Read totals as descriptive summaries of available values. Missing numeric fields can make an enrollment sum incomplete. A low score with missing components does not establish a simple protocol. Phase distributions describe registry labels, including combined labels, rather than classifier predictions.

Fixes. If the page stays empty, check the open backend terminal, health endpoint, API token and current dataset. If it shows the wrong cohort, switch the global selector and verify the dataset ID. If predictions are unavailable, train the appropriate task for the matching dataset and source hash. Do not use an artifact from a different cohort. If an old compiled UI appears, rebuild after source changes, restart and refresh.

The hosted preview keeps the fixture dataset. Importing or training real data belongs to the full app. Inspect `docs/VERIFICATION.md` for which browser and backend combinations were actually tested in the release.

8. Study Explorer - operate and test

Purpose. Search, filter, page, inspect and compare canonical trial records without changing them.

Operate. Search by part of an NCT ID or title. Apply the phase and status selectors. Sort by ID, enrollment or title. Use Next/Previous to view the full dataset. Open a study ID for source fields and prediction evidence. Select two or three studies, then press Compare. Export CSV when needed.

Explorer pages contain up to 25 records. Selectors in several other screens use the first 100 records plus records opened in Explorer. To use a later record in a scenario or model comparison, find and open it here first. The Explorer itself can page through the whole cohort.

Acceptance checks.

Action	Expected result
Search an existing ID fragment	Matching IDs/titles; count changes
Apply phase filter	Only the selected recorded phase appears
Next then Previous	Range and rows change and return correctly
Open a record	The same ID/title and canonical fields appear
Select two studies and Compare	Side-by-side descriptive fields and PCI
Export CSV	Correct cohort; spreadsheet formula-prefix escaping

Clear filters if an expected ID disappears. Recorded phase labels are exact dataset values; a combined label need not match a single-phase filter. Missing values are information, not permission to fill fields manually with guessed numbers.

Fixes. A search with no results can be a valid result. Check spelling, active dataset and all filters. A record beyond the first 100 may need to be opened here before it appears in another screen's selector. If a detail prediction is unavailable, read the reason and verify that a compatible locally generated model exists for the selected dataset. If export fails, check connection and token rather than copying sensitive content into browser developer logs.

Comparison is descriptive. Differences in enrollment or PCI do not establish different treatment benefit, trial success probability or causal duration effects.

9. Complexity Lab - operate and test

Purpose. Explore the fixed-reference Protocol Complexity Index (PCI) and sensitivity to enrollment or weights. This score is distinct from training-relative research PCI.

Operate. Select a study. Read its reference score and six component contributions. Choose Reference, Equal or Custom weighting. For Custom, move a component slider; weights are normalized to sum to 100. Change Enrollment adjustment, observe the scenario score and export its JSON. Use Reset scenario to restore reference weights and zero adjustment.

The source record remains unchanged. An exported Complexity Lab sensitivity result is separate from a persisted Digital Twin scenario. A score change is arithmetic sensitivity; it does not establish the causal effect of an amendment or changed enrollment on duration. Reference PCI rounds the exact decimal total half-up once at two places and classifies the rounded score.

Acceptance checks.

Action	Expected result
Reference mode	Weights/ceilings match `configs/formulas.json`
Equal mode	Each of six components has one-sixth of 100 weight
Custom mode	Displayed normalized weights sum to 100, allowing display rounding
Increase enrollment under its ceiling	Enrollment contribution and total respond
Increase enrollment past its ceiling	Contribution is capped at its weight
Reset	Reference mode, original enrollment and score return
Export scenario	JSON records ID, mode, assumptions and scenario score

Missing PCI inputs contribute zero in this descriptive view and are disclosed. An imported negative/nonfinite numeric value is rejected; scenario controls do not justify accepting malformed source data. Read Formula Studio and the formula specification for the exact convention and category boundaries.

Fixes. If a later study is missing from the selector, open it in Explorer. If the score seems unexpectedly low, inspect missing components and caps before treating it as a model bug. The custom all-zero slider state uses a documented equal-weight fallback; it must not create division by zero. If a change is visually absent at the ceiling, the cap can explain it. Prediction outcomes cannot be inferred from PCI alone.

10. Experiments - operate and test

Purpose. Fit comparable duration regressors and prolonged-risk classifiers on one eligible cohort with chronological outcome-aware splits. All seven default branches share the same retained study IDs.

Operate. Load a real dataset. Read readiness, then click Run experiment. Status progresses queued/running to completed or failed; history polls about every five seconds. Select a run, review warnings and select an evaluation split/model. Inspect duration scatter, risk reliability and per-phase errors. Download the manifest and held-out CSV. Use Compare a study prediction to request a specific branch or the recommended risk branch.

Branch	Role
`median_baseline`	Training-only median duration and prevalence
`structured`	Structured linear Ridge/logistic pair
`text`	TF-IDF linear pair
`multimodal`	Structured plus TF-IDF linear pair
`structured_extra_trees`	Structured nonlinear Extra Trees pair
`structured_random_forest`	Structured nonlinear Random Forest pair
`structured_gradient_boosting`	Structured Gradient Boosting pair, bounded tuning

Acceptance checks. A completed run has source identity, configuration, split IDs, actual fitted metrics and allowlisted downloadable evidence. The seventh branch appears when advanced models are enabled. Tuning selects parameters and recommended risk model; calibration has a separate role; test never selects a model. Prediction displays provenance and cohort membership. Training-row predictions are in-sample. Primary probabilities and classification metrics remain raw; a separately fitted calibrated probability is nullable and explicitly identified.

Duration eligibility requires completed status, exact positive observed duration and ACTUAL completion, with at least 30 eligible records and adequate retained temporal partitions. These are software guards, not a scientific sample-size recommendation. Read the exact current readiness response and `docs/V4_MODELS.md` for partition support rules.

Fixes. A 409/busy response means the shared training slot is occupied; wait. A 422/readiness error means improve cohort dates, labels or support rather than bypassing guards. Failed or interrupted runs remain history; start a new run. Undefined AUROC/AP in a one-class split remains a dash/null. A recommended risk branch need not have the lowest duration error. No branch is guaranteed to improve performance.

11. Phase Academy - operate and test

Purpose. Learn the objectives of optional Phase 0 and Phases I-IV through conceptual native diagrams. This learning screen is independent of fitted models and an API key.

Operate. Select a phase tab and read its question, purpose, illustrative scale and primary-source link. Choose Participants, Study design, Observations or Review. Use Step, Play/Pause, Reset and speed. Complete the five-question quiz, read feedback, move to the next question and reset when finished. Open a matching workspace study as an example of a recorded phase label.

Acceptance checks.

Action	Expected result
Phase III tab	Phase III lesson and comparison-arm illustration
Step	One stage advances; displayed explanation changes
Play	Stages progress and playback stops after Review
Pause/Reset	Playback stops; Reset returns to first stage
Tab focus + Left/Right/Home/End	Phase tab and lesson change with keyboard
Quiz answer	Correctness feedback and explanation appear
Reduced motion enabled	Automatic playback disabled; manual steps remain usable

Participant symbols are illustrations, not enrollment counts, outcomes, success rates or recruited patients. Study designs vary; a simplified lesson is not a protocol specification. Early Phase 1 is a registry label related to exploratory Phase 0 concepts. Combined phases and N/A require their own interpretation and are not forced into a single stage.

Matching studies come from the loaded workspace records used by the interface. No matching example is a valid state; the lesson still works. Keep the original registry phase visible and do not replace it with a classifier estimate.

Fixes. If Play is disabled, inspect the operating system/browser reduced-motion setting; Step is intentionally retained. If a narrow viewport needs horizontal scrolling, focus the diagram region and scroll. If a source link cannot open, the lesson remains local; internet is needed for external reading. If phase examples are absent, check dataset phase labels and open additional records in Explorer.

12. Phase Model - operate and test

Purpose. Compare a trial's reported registry phase with a separate supervised educational consistency model. This task does not predict phase advancement or assign a regulatory phase.

Operate. Select a real dataset and read readiness. Click Train phase model, wait for completed status, and inspect validation/test accuracy, balanced accuracy, macro-F1 and class support. Read the confusion matrix: rows are actual labels; columns are predicted labels. Select a study and click Compare phase. Export the phase evidence.

At least 40 eligible rows, exact start dates, supported single-phase labels, 20/5/5 train/validation/test rows and two training classes are required. Completion outcomes are not required. Combined labels, N/A, unknown and missing phases are excluded with reasons. The model uses allowed numeric fields and phase-redacted text; the phase target itself is not a feature.

Acceptance checks. Reported phase remains visible next to predicted phase. The probability catalog contains five canonical labels; scores are labeled uncalibrated. The manifest has `task: phase`, phase-specific filenames, source identity, split IDs and artifact version 1. Baseline and classifier evidence remain separate from duration results. A class absent from training cannot be learned.

The starter has no Early Phase 1 examples. A zero model probability for that unseen class is a training-support limitation, not proof that the phase is impossible. Macro-F1 uses the documented actual/predicted label denominator; inspect support before comparing values across splits.

Fixes. If training is refused, inspect exclusions, exact dates and early training-class support. If another task is running, wait for the shared slot. If prediction is unavailable, confirm a completed matching-source phase run and trusted intact manifest/artifact. Corrupt, mismatched or incompatible artifacts are intentionally refused. A confident score does not make a disagreement clinically authoritative; inspect held-out errors and source provenance.

See `docs/PHASE_MODEL.md` for redaction rules, normalization, class metric definitions and Python/HTTP contracts. The retained phase artifact format is separate from the changed duration artifact format in v4.

13. Data Manager - operate and test

Purpose. Register immutable sources, select datasets and inspect data quality.

Operate. Click Load starter cohort for an offline real-data start. To import your own table, download the CSV template, choose CSV/Parquet, enter an optional name and press Validate & import. For a live source, enter a ClinicalTrials.gov search expression, choose 50-1000 maximum studies and press Import registry records. Select Use dataset in the registry table. Review missingness, warnings and checksum.

Acceptance checks.

Action	Expected result
Load starter	800 public records registered and selected
Load starter again	Same source selected; no duplicate source created
Import a valid unique-ID file	Dataset, row count, provenance and quality appear
Import malformed/duplicate/ nonfinite data	Safe rejection; no partial dataset registered
Switch datasets	Overview and subsequent requests use selected ID
Inspect quality	Missingness counts agree with canonical source values

Default upload limits are configured server-side: 25 MiB, up to 100,000 dataset rows and up to 20,000 training rows. Read `.env.example` and `backend/app/config.py` before changing bounds; hardware memory and experiment cost still matter. A data import can pass validation while failing outcome eligibility for modeling.

Raw upload bytes, canonical data and source metadata are preserved. CSV column names must be unique and row widths must match the header. Partial ISO dates are supported as data but excluded from exact-duration training. Do not replace a missing type with ACTUAL unless supported by real source evidence.

Fixes. A 413 response means the upload exceeds configured size; reduce or deliberately configure a suitable workflow. A 422 means inspect validation errors rather than removing safeguards. A 502 during registry import is an upstream/network failure; retry later without assuming fixtures were imported. A successful fresh registry import can differ from the frozen starter. On the preview, import controls disclose that the full backend is required.

14. AI Assistant - operate and test

Purpose. Ask independent questions through a configured AI provider with bounded study or dataset context. The assistant does not fit models, browse websites or create new validated research predictions.

Operate. Configure the server key as described earlier and restart. Choose the current dataset summary or a study. Ask a concrete question, such as: "Which source fields are missing, and how does that limit interpretation of this descriptive PCI?" Press Send and check the reply against the selected record and manifest.

Earlier visible chat turns are local display context; each provider request is independent. Restate essential context in a follow-up question. Dataset context is bounded, so a reply should not be interpreted as having reviewed every row in a large cohort.

Acceptance checks.

Action	Expected result
No server key	Clear unavailable/configuration message; core app remains usable
Valid own key + permitted model	Real reply after user-initiated Send
Only one configured key	Auto selects that matching provider
Several configured keys	Auto follows the declared priority
Invalid key or blocked quota	Auto can try the next provider; errors never disclose keys
Direct provider selection	Only that provider is contacted
Provider refusal or safety block	No automatic rerouting
Successful answer	Actual provider and model appear above the reply
Select a study	Reply context identifies the selected study
Ask for absent fitted outcome	Response distinguishes missing evidence from computed results
Switch dataset/context	New request uses the new selected context

A controlled provider-contract test does not prove a live account works. Check current `docs/VERIFICATION.md` for release boundaries. Never claim provider authentication from `assistant_enabled: true`; that flag reports configuration only.

Fixes. If configured but failing, check permissions, model access, quota and network locally. If still using old settings, restart the backend. If a study is absent from the selector, open it in Explorer. If you set a shared app token, enter that token in Connection settings. Do not put a provider key there. Verify any assistant statements about metrics, study design or source provenance independently; the model's instructions do not guarantee correctness.

15. Research Hub - operate and test

Purpose. Bring the selected study, descriptive PCI, model availability, similar-record retrieval, phase context and source provenance into one researcher view.

Operate. Choose an Active study, inspect its source fields and PCI, then open Explain the model evidence if a compatible local model is available. Read Find protocols with related text and the retrieval method. Source record opens preserved values; Formula Studio, Inspect cohort validation and Explore a scenario lead to related screens. Evidence JSON downloads the selected-study packet. Review the dataset ID, checksum and fixture/source labeling before exporting.

Similarity uses **TF-IDF cosine** over allowed narrative text. Self is excluded. Where same-phase filtering is selected, it compares normalized recorded phase exactly, including combined labels. The retrieval scope is the selected canonical dataset. It is descriptive text similarity, not a treatment match, clinical equivalence, safety or success score.

Phase context reports dataset summaries, such as the count and median enrollment/PCI for the recorded phase. It is not a model prediction or a causal reference group. Missing data and small groups limit interpretation.

Acceptance checks.

Action	Expected result
Select a record	All cards refer to the same ID and dataset
No compatible model	Duration/phase remain unavailable with reason
Review similarity	Selected ID is absent; method/scope is disclosed
Apply same-phase retrieval where exposed	All matches share normalized recorded phase
Inspect provenance	Dataset ID, SHA-256, source and fixture status agree
Review recruitment/amendment risks	Unavailable without curated labeled task/models

Fixes. No similar results can be valid for sparse text or phase scope. Empty text does not justify fabricating a score. If a prediction disappears after dataset selection, verify source identity and task rather than attaching an unrelated model. If you loaded real data but still see fixture status, check the global selector. Source outcomes and IDs remain immutable.

The Hub connects evidence already supported by the application. It does not invent recruitment failure probabilities, amendment probabilities or prospective validation. Use Digital Twin for explicitly assumed planning arithmetic.

16. Digital Twin - operate and test

Purpose. Create an immutable hypothetical protocol scenario and a transparent enrollment plan while preserving the original study. "Digital Twin" names a planning workspace, not a clinically validated patient simulation.

Operate. Select a Source study. Name a scenario, edit permitted protocol features and set sites, monthly recruitment rate per site, startup months and follow-up months. Press Compute & save scenario. Review base versus scenario PCI, planning duration and model changes when available. Select a saved history name to inspect that immutable result; Scenario JSON exports it. Evidence JSON exports the selected-study packet. Restore inputs returns source design values and initial editable planning assumptions of 10 sites, rate 2, startup 3 and follow-up 6.

Permitted overrides are enrollment, arms, primary/secondary outcomes, criteria count, text complexity and phase. Observed outcomes, dates, statuses, IDs and source text are immutable. Blank optional design fields preserve source values. Numeric inputs must be finite and within validation bounds; sites and per-site rate must be positive. A missing target must be explicitly supplied rather than assumed zero. A deliberately entered zero target is allowed.

Bounds: enrollment 0-10,000,000; arms 0-1,000; primary/secondary outcome counts 0-10,000 each; criteria count 0-100,000; text complexity 0-10,000; sites 1-100,000; rate 0.000000001-1,000,000; startup/follow-up 0-1,200 months. Counts/sites are whole numbers. See `docs/V4_RESEARCHER.md` for the field contract.

Digital Twin - planning, journey and acceptance

Known-answer test. Set enrollment to 120, sites to 10, rate to 2 participants per site per month, startup to 3 months and follow-up to 12 months. Recruitment is `120 / (10 * 2) = 6 months`. Total planned duration is `3 + 6 + 12 = 21 months`. These are explicit planning assumptions, independent of fitted duration predictions in days.

Planning known answer: 21 months under stated assumptions



Startup Recruitment Follow-up

```
120 / (10 sites * 2 participants/site/month) = 6 recruitment months
```

The native Research Journey explains Protocol, Startup, Recruitment, Follow-up, Analysis and Closeout. Protocol, Analysis and Closeout are conceptual/unmeasured; only the three quantitative stages form the proportional duration bar. Play/Pause, Step, Reset, speed and stage buttons work with a keyboard. In the focused journey panel, Space toggles playback, Left/Right changes stage and Home resets. Playback stops after Closeout. Reduced motion disables autoplay/speed but retains manual control.

Acceptance checks. The saved scenario has an ID, creation time, name, base and changed values, source provenance and evidence digest. History survives restart in the local app; fixture preview history is page memory until refresh. Reopen the original study: its fields are unchanged. A zero/negative rate is rejected. Unavailable fitted outcomes remain null, and model scenario estimates are labeled hypothetical with cohort membership and warnings. Journey numbers and bar widths agree with the live planning arithmetic.

Fixes. For missing target, supply a supported enrollment assumption; do not guess from an unrelated study. For unavailable model changes, fit a matching-source duration model. Out-of-range inputs are validation errors; reduce them or review bounds deliberately. A fixture preview can demonstrate arithmetic but cannot run the Python training backend or manufacture a fitted counterfactual.

Constant rates, concurrent sites and additive stages are simplifying assumptions. No startup, site activation, dropout, nonlinear enrollment curve or recruitment risk has been learned from curated outcomes here.

17. Validation Lab - operate and test

Purpose. Explain a real run's cohort, exclusions, chronological partitions, tuning, calibration and held-out evaluation. It is an evidence viewer, not prospective external validation.

Operate. Complete a local duration run, choose Completed duration run and inspect its manifest. Refresh evidence after a new fit. Use Play/Pause, Step, Reset and speed to follow Train, Tune, Calibrate and Test. Compare visible counts with exported split IDs. Examine outcome-availability purging, raw/calibrated probability semantics, interval support and warnings. Export the evidence for audit. Before a run, the role explanation works without fabricated counts or metrics; reduced motion retains manual controls.

Chronological roles - diagram is explanatory, with no invented counts



Whole start-date groups remain intact. Outcome availability is purged at cutoffs.

Earlier original-validation groups tune; later groups calibrate. Inspect actual manifest IDs.

Acceptance checks.

Action	Expected result
No local run	Honest empty/unavailable evidence state
Choose completed run	Actual manifest/source identity, not roadmap percentages
Inspect timeline	Train, tuning, calibration and test roles are distinct
Compare counts	Display agrees with manifest IDs and exclusions
Examine recommendation	Tuning risk Brier selects; test does not
Calibration unsupported	Explicit unavailable state with reason
Undefined metric	Null/dash rather than fabricated zero

The original validation partition is split chronologically by whole start-date groups: earlier groups tune parameters; later groups calibrate uncertainty. Calibration is not used to select hyperparameters or the recommended risk branch. The selected pipelines remain trained on training data rather than refitting with held-out rows.

Finite-sample conformal intervals depend on calibration residual support and exchangeability. Chronological drift can violate that assumption. Nominal coverage is not a guaranteed achieved coverage for future trials. Probability calibration requires sufficient both-class support; if unavailable, the output must retain explicit raw semantics.

Fixes. If counts are small, inspect date groups, exclusions and purged outcomes rather than forcing a split. If the lab shows an old run, choose a current duration artifact version 3 run. If a

chart is empty, inspect null metric and support information in the manifest. Passing the software visualization check does not validate the model externally or clinically.

18. Formula Studio - operate and test

Purpose. Make formulas, units, weights, assumptions and known answers inspectable. The shared configuration is `configs/formulas.json`; the detailed contract is `docs/FORMULA_SPECIFICATION.md`.

Operate. Select a Protocol and inspect the six weighted PCI terms. Use Play terms/Pause, Step term, Reset terms and speed to see terms accumulate. Compare the final result with Complexity Lab in Reference mode. Read the metric and planning equations. Use known inputs to check both displayed arithmetic and the source tests.

Acceptance checks.

Formula	Input	Expected result
PCI, all inputs at zero	Six zero components	0, Low
PCI, all inputs at their ceilings	Six capped components	100, High
PCI, all inputs at half ceilings	Six half-weight terms	50, Medium
MAE	Actual `[1,2,3]`, predicted `[1,2,4]`	`1/3`
RMSE	Same arrays	`sqrt(1/3)`
R2	Same arrays	0.5
Binary Brier	Labels `[0,1]`, scores `[0.2,0.8]`	0.04
Cosine	Vectors `[1,0]`, `[1,1]`	`1/sqrt(2)`
Planning	Target 120; 10 sites; rate 2; startup 3; follow-up 12	Recruitment 6; total 21 months
Conformal rank	Alpha 0.1; n=10 / n=3	Rank 10 / rank 4 (unbounded)

R2 is unavailable when target variation is insufficient; AUROC/AP are unavailable with a one-class evaluation target. A cosine score is descriptive and is not automatically calibrated to clinical similarity. Formula examples use known mathematical answers and are not fabricated trial metrics.

Fixes. If PCI differs, first match dataset, study, weighting mode and missing-value convention. Compare unrounded totals before display rounding. A custom sensitivity score is not the reference score. If an animation is disabled by reduced motion, use manual steps; the numerical result must remain accessible. If changing the source configuration, rebuild the frontend and rerun Python/JS parity checks so the browser and server consume the same rules.

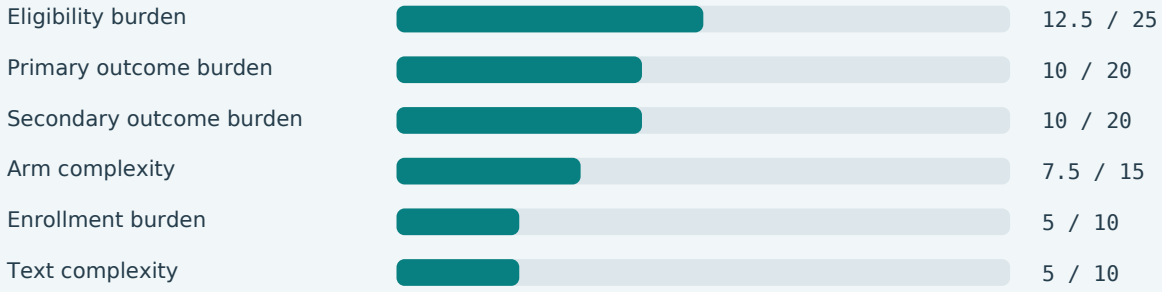
Use the final `docs/VERIFICATION.md` for executed formula-test evidence. Avoid treating a formula animation or a passing arithmetic test as proof of a clinically meaningful index.

19. Exact PCI, metric and planning formulas

Fixed-reference PCI is `sum(weight_i * clip(value_i / ceiling_i, 0, 1))`. Reference weights sum to 100. Sum unrounded exact decimal-rational contributions and round the total once, **half-up**, to two places. Classify the rounded total: Low below 40, Medium from 40 through 70, and High above 70. Thus 39.995 becomes 40.00/Medium, 70.0049 becomes 70.00/Medium and 70.005 becomes 70.01/High. Missing/invalid/nonfinite components contribute zero and are disclosed; finite negative source values clip to zero. Imported source validation is stricter. Zero contribution is not evidence of absent burden.

Component	Canonical field	Ceiling	Weight
Eligibility burden	`criteria_count`	40	25
Primary outcome burden	`primary_outcomes`	10	20
Secondary outcome burden	`secondary_outcomes`	20	20
Arm complexity	`number_of_arms`	8	15
Enrollment burden	`enrollment`	1000	10
Text complexity	`text_complexity`	20	10

PCI known answer: every value at half its ceiling gives 50 points



Bars use one common 0-25 contribution scale. These are formula inputs, not trial outcomes.

The criteria count and text complexity fields are proxies, not validated medical concepts. The starter downloader does not invent missing extraction fields. Training-relative PCI uses training-fitted ranges and medians and is separately recorded in artifacts; do not mix it with fixed-reference PCI. The decimal correction preserves five v3 fixture scores; some real starter scores differ by 0.01 from older binary rounding. This does not imply changed observed outcomes or improved models.

Metric and planning definitions

Quantity	Exact definition	Interpretation
MAE	`mean(abs(y - prediction))`	Average absolute error, duration days
RMSE	`sqrt(mean((y - prediction)^2))`	Larger errors receive more weight
R2	`1 - sum(error^2) / sum((y - mean(y))^2)`	Can be negative; unavailable for constant target
Binary Brier	`mean((probability - binary_label)^2)`	Lower is better; not discrimination alone
Cosine similarity	`dot(a,b) / (norm(a) * norm(b))`	Text-vector alignment; handle zero vectors explicitly
Recruitment months	`target / (sites * rate_per_site_per_month)`	Positive sites/rate required
Planned total months	`startup + recruitment + followup`	Additive assumed stages

Days and months are separate units. The planning workspace must not subtract its months directly from a learned duration in days without an explicit conversion policy. A probability delta is a difference in model scores, not causal evidence.

20. v4 models, calibration and uncertainty

The duration label is the observed number of days from exact start to ACTUAL completion. "Prolonged" means strictly above the training cohort's 75th-percentile duration. That threshold is learned from training only and is then fixed for later partitions. It is not an externally validated clinical threshold.

Whole start-date groups are chronological. Training outcomes must be available before the original validation cutoff, and original validation outcomes before the test cutoff; unavailable outcomes are purged and exported. Original validation groups divide near the halfway group into earlier tuning and later calibration. At least 10 later calibration rows and one earlier tuning row are required; otherwise all original validation rows tune and calibration is empty. Existing minimum cohort guards remain. In the backward-compatible manifest, `splits.validation` means tuning and `splits.calibration` is separate. Read `validation_partition` and `docs/V4_MODELS.md` for actual IDs/counts and fallback reasons.

Numeric imputation/scaling, categorical vocabulary and TF-IDF vocabulary/IDF are fitted on training only. Linear candidates and advanced branches are selected on tuning metrics. Gradient Boosting uses 64 estimators, depth 3, learning rate 0.05 and leaf-size candidates 2/5. Extra Trees/Random Forest use 64 trees, depth 12 and leaf candidates 2/5. Linear grids are Ridge alpha 1/10/100 and logistic C 0.1/1/10. More capacity does not guarantee improvement. CLI `--config configs/research.json` can disable advanced branches; GUI training uses `ExperimentConfig` defaults, not that CLI file. Change GUI defaults deliberately in code and rerun checks when needed.

Recommendation uses **raw tuning Brier**, exact ties by model name. It is not a claim that the same branch minimizes duration RMSE. Primary prediction `probability`/`raw_probability`, CSV `prolonged_probability`, primary classification/reliability metrics and recommendation all remain **raw**. Separate nullable `calibrated_probability` and calibrated metric fields disclose the optional transformation. Original model explanations describe raw fitted behavior. Test outcomes assess, but never select or fit, these quantities.

Split-conformal duration calibration uses absolute residuals from the dedicated calibration subset. For nominal miscoverage `alpha`, the finite-sample rank is `ceil((n + 1) * (1 - alpha))`; use that ordered observed residual without interpolation. Default alpha is 0.1. A finite quantile q gives `[max(0, predicted_days-q), predicted_days+q]`. If rank exceeds n, JSON retains `status=available`, `bounded=false` and null quantile/bounds: the mathematical interval is unbounded. Insufficient calibration instead has `status=unavailable` with a reason. These intervals rely on exchangeability and can lose coverage under temporal shift.

Probability calibration requires at least 10 calibration rows and two labels of each binary class. A deterministic regularized sigmoid fits raw-probability logits on calibration only. Under-supported data leave calibrated scores null while raw predictions remain available. Calibration-set diagnostics are in-sample for that sigmoid; only test diagnostics use untouched labels. The starter's limited validation positives are an important boundary. Out-of-range numeric and unseen-category checks describe support/extrapolation; they are not a learned probability of being out of distribution.

Duration v4 artifacts use **version 3**. Phase artifacts retain **version 1** under their own task contract. Source identity and trusted local provenance are checked in addition to version. Checksums are integrity checks, not signatures for accepting untrusted serialized models.

21. How to interpret evidence honestly

Registry snapshots can contain fields revised after trial start. Temporal outcome purging prevents a specific label-availability leak; it does not prove baseline feature provenance. The bundled latest-state cohort remains retrospective unless acquisition/history evidence establishes baseline values.

Evidence	What it supports	What it does not establish
Software test/fixture	Contract and arithmetic behavior	Clinical validity or real model accuracy
Fixed PCI	Explicit descriptive weighted index	Trial success, recruitment failure or clinical burden standard
Text similarity	Retrieval within selected dataset	Clinical equivalence or treatment recommendation
Held-out temporal metric	This fitted run under its cohort/snapshot assumptions	Universal or prospective performance
Model contribution	Association in fitted predictions	Causal effect of a protocol change
Scenario prediction	Hypothetical feature-query response	A randomized counterfactual outcome
Conformal nominal level	Method under support/exchangeability conditions	Guaranteed coverage under chronological shift

The starter is a small completed-only convenience subset. Exact dates exclude many records and can favor particular years/study types. It has no censored/ongoing outcomes for the duration task. Low positive counts make discrimination and calibration fragile. Per-phase metrics can be unstable or undefined; inspect support and warnings before making comparisons.

Linear branches show exact fitted contributions where supported. Tree impurity importance is global and can favor certain features; it is neither local SHAP nor causal proof. Optional SHAP requires an appropriate original training background and is a separate dependency. No unexecuted optional method should be described as verified evidence.

Recruitment failure and amendment outcome probabilities remain unavailable without curated labels and validated task models. Phase scores do not assign regulatory phases. Planning arithmetic does not learn recruitment rates. The assistant cannot fill these evidence gaps by generating plausible text.

For a publication or decision, curate baseline/history provenance, prespecify a cohort and analysis, obtain sufficient labels, assess independent external or prospective data, report uncertainty and document biases. These are future research requirements, not capabilities proved by the release ZIP.

22. Export, evidence packets and backups

Study Explorer exports CSV. The full app's report button downloads self-contained descriptive HTML; open it and use Print > Save as PDF if desired. CSV cells are escaped against spreadsheet formula prefixes, and descriptions are HTML-escaped. A browser preview print/export can describe only its fixture scope.

Experiment evidence includes manifests, metrics, held-out predictions, cohort exclusions and temporal exclusions. Duration files use `manifest.json`, `metrics.json`, `held_out_predictions.csv` and `models.joblib`. Phase files have a `phase_` prefix. GUI download endpoints allowlist evidence JSON/CSV; binary models are not general upload/download interchange.

Research evidence packets contain bounded selected-study workspace/scenario information, generation time, source scope and a SHA-256 digest. Compare the packet's source identity to the active dataset. A digest makes changes detectable; it does not prove authenticity or validate science. Keep each packet with its relevant run manifest.

Backup steps.

1. Stop the server with Ctrl+C and confirm it has exited.
2. Copy the complete `data/workbench/` folder, immutable `data/raw/`, relevant processed cohorts, `models/`, `results/` and configuration.
3. Preserve public source metadata, hashes and research manifests alongside artifacts.
4. Keep `.env` and database credentials private in a separate protected backup.
5. If using PostgreSQL, also back up the database with your normal database tools. Files remain necessary.
6. Test restoration in a new installation/folder, check row counts and history, then run matching-source predictions or retrain.

Do not copy only `registry.sqlite3` while the app is writing. Do not load joblib/pickle from an untrusted sender: serialized Python artifacts can execute code. Preserve trusted local directories intact and retrain if integrity/version/source checks fail.

Acceptance check. After a clean stop and restore, dataset IDs/counts and scenario history match the backup. Current compatible local experiments are recognized. Source hashes agree. A mismatched model stays unavailable rather than being force-loaded.

23. Upgrade v3 to v4 safely

1. Stop v3 and back up the whole persistent store, source/processed files, research results, configuration and private `.env`.
2. Keep the delivered v3 ZIP or installation unchanged as an archive.
3. Extract v4 into a **new folder**. Do not overwrite your only working copy.
4. Run v4 setup with Python 3.12. Use its new environment; do not copy old `.venv` or `node_modules`.
5. Copy your backed-up data store and private settings into v4. Compare new `.env.example` settings; retain your chosen private values deliberately.
6. Start once. Check health version, dataset counts, source identities, import quality and existing history.
7. **Retrain duration/risk experiments.** v4 duration artifact version 3 replaces v3 version 2 because partition and uncertainty semantics changed. Old runs can remain history but are not force-loaded.
8. Phase format remains artifact version 1. Reuse only a trusted local phase artifact that still satisfies matching task, run, version, source identity and integrity checks. Retrain if compatibility is refused or you want a fresh reproducible run.
9. Run doctor, dependency checks, tests, build and screen acceptance checks. Back up the verified v4 store before further research.

```
.venv/bin/python scripts/doctor.py
.venv/bin/python -m pip check
.venv/bin/python scripts/run_smoke_test.py
.venv/bin/python scripts/verify_release.py
```

Run `verify_release.py` from a fresh extracted package before editing files; intentional edits can change release checksums. On Windows use `.venv\Scripts\python.exe` instead of `.venv/bin/python`.

Rollback by stopping v4 and using the preserved v3 installation with its own backed-up v3 store and environment. Do not assume new scenario tables or v4 artifacts are accepted by v3. Never overwrite frozen raw source files to force agreement.

If upgrading from v2, consult the archived runbook and retrain duration artifacts through the current version. An old history record is not a promise of loadable model compatibility.

24. Troubleshooting by symptom

Symptom	Check and fix
Python launcher missing	Install Python 3.12 with launcher; reopen terminal; run `py -3.12 --version`
Setup cannot install	Read first pip error; check proxy/network/disk; retry with the correct interpreter
Required module missing	Use `.venv` Python to install requirements; do not use another environment's pip
Built interface missing	Install Node.js 22; `npm ci` then `npm run build` in `frontend/`
Port 8000 in use	Stop other process or set `API_PORT=8001`; restart and use printed URL
Cannot reach API	Keep backend terminal open; load health; verify URL/port and network
Unauthorized	Match server `API_BEARER_TOKEN` with browser Connection settings
Configured assistant fails	Check own key/model/project permissions/quota privately; restart after edits
Import rejected	Fix duplicate IDs, row shape, required columns, finite numerics and dates
Registry import fails	Inspect safe upstream error; retry later; no sample substitution occurs
Duration readiness fails	Inspect exact ACTUAL outcomes, eligibility, date groups and purging
Phase readiness fails	Inspect exact start dates, single phases, split sizes and training classes
Training busy	Wait for the one shared duration/phase slot
Job failed after restart	Review failure; launch a new experiment
Prediction unavailable	Use a completed compatible matching-source local run; retrain if needed
Calibration unavailable	Review both-label/support guards; retain explicit raw score semantics
AUROC/AP is a dash	One-class target is mathematically undefined; obtain/report proper support
Scenario target missing	Enter a supported enrollment override; never assume missing means zero
Scenario rate invalid	Sites and rate must be finite positive values within supported bounds
Study absent in selector	Find/open it in Explorer; selectors include first 100 plus opened studies
Play disabled	Reduced-motion preference intentionally blocks autoplay; use Step
Source/model digest mismatch	Preserve evidence; restore trusted matching files or retrain
UI change invisible	Rebuild interface, restart server and refresh browser

When reporting a bug, include the app version, operating system, safe error message, screen/action, dataset ID and whether it is a fixture. Include redacted readiness/manifest evidence where useful. Never include `.env`, keys, signed URLs, database passwords or private trial text. A screenshot should show the relevant state, not credentials.

25. Developer checks and every-section acceptance

Use the virtual-environment interpreter. Install development requirements before Python tests:

```
.venv/bin/python -m pip install -r requirements-dev.txt
.venv/bin/python -m pytest -q
.venv/bin/python scripts/doctor.py
.venv/bin/python -m pip check
.venv/bin/python scripts/run_smoke_test.py
```

Windows equivalents:

```
.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.venv\Scripts\python.exe -m pytest -q
.venv\Scripts\python.exe scripts\doctor.py
.venv\Scripts\python.exe -m pip check
.venv\Scripts\python.exe scripts\run_smoke_test.py
```

Node.js 22 is optional for users and required to rebuild source frontend:

```
cd frontend
npm ci
npm run build
```

For hot reload use `npm run dev` with the API on port 8000. Vite proxies `/api` to the backend. A separate deployed frontend may use public `VITE_API_BASE_URL=https://YOUR-API/api/v1`; that configuration is visible to browsers and must never contain secrets.

Every-screen release acceptance checklist

Section	Minimum acceptance
Overview	Fixture/real counts, source label, unavailable prediction honesty
Explorer	Search, filter, page, detail, compare and CSV
Complexity	Reference/equal/custom, cap, reset and JSON
Experiments	Readiness, real fit, seven enabled branches, evidence and matching prediction
Academy	Phase tab, step/play/reset, quiz and reduced motion
Phase Model	Separate real fit, source label, scores, class support and exports
Data	Starter idempotence, valid import, invalid atomic refusal and provenance
Assistant	Safe no-key state; controlled errors; own live key only if supplied
Hub	Source identity, self-excluded retrieval and honest risk availability
Digital Twin	120/10/2 planning = 6 and 21; source immutable; persisted history
Validation	Real partition counts, calibration role and null metric handling
Formula	Known answers, shared constants, Python/browser agreement

Record executed checks in `docs/VERIFICATION.md`, with environment and evidence. A passing unit suite does not imply Windows, Docker, mobile, live provider calls, Neon or prospective validation was exercised. Run the actual launcher and full local API as well as a fixture preview when validating a release.

26. Research CLI and AACT workflow

For a frozen AACT flat-file workflow, obtain the official dated snapshot through `https://aact.ctti-clinicaltrials.org/downloads/snapshots?type=flatfiles`. Preserve the archive unchanged, its actual date and independently verified SHA-256. A PostgreSQL dump is not the adapter's flat-file input.

Extract to a new directory. The adapter verifies table headers and parseability, aggregates one-to-many tables before joins and preserves provenance. Missing optional tables produce missing values rather than fabricated defaults. Results-table outcomes are excluded from model features.

```
.venv/bin/python scripts/verify_aact.py --path data/raw/aact_snapshot
.venv/bin/python scripts/build_features.py --input data/raw/aact_snapshot --output
  data/processed/cohort-v1 --snapshot-date YYYY-MM-DD
.venv/bin/python scripts/train_models.py --input data/processed/cohort-v1/trials.csv --output
  results/cohort-v1 --config configs/research.json --metadata
  data/processed/cohort-v1/source_metadata.json
.venv/bin/python scripts/evaluate_models.py --experiment results/cohort-v1
```

Replace the snapshot date with the actual source date. Use each script's `--help` for current flags. Supply new output directories; prior artifacts are not overwritten. Independent CLI experiments are not automatically GUI history. Import the canonical CSV and train through the GUI when you need local registered predictions.

The starter can be trained by CLI with new output folders:

```
.venv/bin/python scripts/train_models.py --input data/starter/trials.csv --output
  results/starter-duration-v4 --config configs/research.json --metadata
  data/starter/metadata.json
.venv/bin/python scripts/train_phase_model.py --input data/starter/trials.csv --output
  results/starter-phase-v4 --metadata data/starter/metadata.json
```

`download_aact.py` accepts an exact authorized HTTPS URL and independently recorded expected hash. If you do not have a trusted expected checksum, acquire and document manually; do not invent one. Avoid storing credentials or signed query strings in source metadata.

Optional `requirements-shap.txt` and `requirements-research.txt` enable separate research modules. BioBERT needs a pinned 40-character revision and explicit model download; it creates embeddings, not an integrated neural prediction head. Cox requires a separately curated event/censoring cohort. See `docs/RESEARCH_METHODS.md` before enabling them.

27. API, authentication and hosting

The local base is `/api/v1`. Health is public. If `API_BEARER_TOKEN` is configured, other routes require `Authorization: Bearer YOUR_APP_TOKEN`. This shared token protects a small trusted workspace; it is not per-user tenancy or role-based access.

Route	Purpose
`GET /health`	Version, health and safe configuration status
`GET /datasets`, `POST /datasets/starter`	Source registry and bundled cohort
`GET /studies`, `GET /studies/{nct_id}`	Canonical records and detail
`GET /readiness`	Duration/phase eligibility and split support
`POST /experiments`, `POST /phase-experiments`	Separate registered training tasks
`POST /predictions`, `POST /phase-predictions`	Matching-source task predictions
`GET /research/workspace`, `GET /research/similar`	Integrated study evidence and retrieval
`POST /research/scenarios`, `GET /research/scenarios`	Immutable scenario creation/history
`GET /research/evidence`	Bounded selected-study JSON evidence packet
`POST /assistant`	Optional bounded provider request; optional provider override

Dataset/study research routes use `dataset_id` and `nct_id` query values; scenarios receive validated JSON. Omitted dataset scope in original routes means the labeled fixture default. Use explicit source IDs for research. See `docs/API_CONTRACT.md` and `backend/app/v4_routes.py` for exact response/schema details.

Full hosting requires persistent disk and a long-running single process. Set HTTPS and the shared app token before exposing it to others. A Vercel fixture/static frontend can run separately; ephemeral functions do not provide the current filesystem registry or in-process training architecture. Production scaling needs durable job queues, object storage, per-user authorization, quotas and migrations.

Neon is optional metadata storage. Install `requirements-postgres.txt`, configure private `DATABASE_URL` with `sslmode=require`, restart and check access. This creates tables but does not migrate an existing SQLite store. Raw/canonical/artifact files remain on persistent disk. No live database connection is implied by supplied configuration.

28. Tools, plugins and v5 development matrix

The workbench does not require your design-tool/plugin accounts at runtime. A requested or installed plugin is not automatically connected, exercised or deployed. `docs/INTEGRATIONS.md` records actual session outcomes; `docs/VERIFICATION.md` records release checks. Preserve that distinction when sharing the project.

Tool/capability	Status and interpretation
OpenAI / Gemini / Claude	Native server adapters; live access requires your matching key
Build Web Apps / Superpowers	Development workflows, not trial-data services
Figma	Editable v4 researcher-workflow handoff; see V4_INTEGRATIONS.md for its link
MagicPath	Historical design context; no v4 runtime dependency
Sites / Vercel	v4 preview published with Sites; existing Vercel project access was blocked. Full Python backend is separate
GitHub	Source/CI configuration supplied; push/remote CI only if explicitly recorded
Neon	Optional metadata adapter; live provisioning/testing only if recorded
Notion / tldraw	Prior handoff status in integrations; no assumed live synchronized workspace
OpenAI Developers / Context7	Documentation tooling status is recorded; not model inference providers
Other model providers	Optional OpenAI-compatible Chat Completions endpoint; key, URL and model required

v5 development matrix - future work

Area	v4 implemented foundation	v5 work requiring new evidence/engineering
Researcher workspace	Hub, planning scenarios, provenance and evidence export	Team workflows, durable collaboration and reviewed permissions
Protocol scenarios	Permitted feature overrides, fixed PCI, optional fitted query	Causal identification and validated counterfactual evaluation
Recruitment/amendments	Explicit unavailable outcome states	Curated labels, task models and independent evaluation
Duration uncertainty	Separate tuning/calibration, support-aware intervals	Larger calibration cohorts and external/prospective assessment
Drift/extrapolation	Training ranges and unseen-category disclosure	Validated drift detectors and operational monitoring
Model families	Seven default comparable duration/risk branches	Evidence-backed new families, controlled ablations and benchmarking
Phase education	Conceptual animation and consistency classifier	Expanded curated phase cohorts and validated learning studies
Data	Frozen 800-row convenience source; import/provenance pipeline	Baseline histories, representative cohorts and research warehouse
Assistant	Bounded multi-provider context and tested routing	Live account validation and governed retrieval
Deployment	Local persistent single process; fixture preview	Queue, object storage, multi-user roles, migrations and audit operations
Scientific validation	Retrospective manifests and limitations	Independent external and prospective validation; no implemented claim

29. Change the project and maintain the manual

Work in a branch and preserve immutable source data. The frontend lives in `frontend/src`; backend integration is in `backend/app`; research models are in `src/clinical_trial_intelligence`; shared formula settings live in `configs/formulas.json`. Review `AGENTS.md`: do not fabricate data, commit secrets, alter raw AACT sources or introduce temporal leakage.

Before a model change, decide the task contract, eligible labels, split strategy, evidence exports and artifact compatibility. Before a formula change, update the shared source, Python/browser parity checks and documentation together. Before adding a provider, implement the backend adapter with bounded context, timeout/error handling and server-only keys; test it separately. No new service is connected by adding a settings field alone.

Run relevant tests, rebuild the frontend, start the actual launcher and test the user workflow. Inspect manifests and known-answer arithmetic. Update `docs/VERIFICATION.md` with exact observed results and boundaries. Regenerate the PDF only after source instructions match implemented behavior.

This guide's editable source is `docs/V4_FULL_GUIDE.md`. Its deterministic layout generator is `scripts/build_v4_guide.py`. The output is `docs/Clinical-Trial-Intelligence-v4-Full-Guide.pdf`. PDF generation requires ReportLab in your chosen Python environment; it is not needed to run the application.

```
python scripts/build_v4_guide.py
pdftoppm -r 90 -png docs/Clinical-Trial-Intelligence-v4-Full-Guide.pdf tmp/pdfs/v4-guide
```

For a release, review every page in a contact sheet and inspect representative dense tables, code and diagrams at full size. Check for clipped lines, split table headers, overflow, missing glyphs, awkward whitespace and incorrect page navigation. The PDF includes chapter bookmarks and a table of contents; regenerate after chapter changes.

Use `README.md` and `START-HERE.md` for quick startup, `docs/RUNBOOK.md` for the compact operating reference, `docs/FORMULA_SPECIFICATION.md` for numerical rules, `docs/V4_MODELS.md` for uncertainty semantics, `docs/V4_RESEARCHER.md` for new researcher screens and `docs/STARTER_DATA.md` for frozen source detail. Consult `docs/STARTER_RESULTS_V4.md` and actual versioned manifests rather than assuming older example scores apply to a new model version. Plugin usage and design links are documented in `docs/V4_INTEGRATIONS.md`.